

Faculty of Engineering

Smart Contract Applications

Mini Compound dApp

Collateralized Lending / Borrowing Platform

Inspired by Compound Finance

Project: Smart Contract Applications

Topic: Collateralized Lending / Borrowing Platform

GitHub: github.com/abdelrahman72-22/mini-compound

Date: May 2026

Network: Hardhat Local & Sepolia Testnet

1. Problem Understanding

1.1 What is the Problem?

In traditional finance, borrowing money requires a bank, extensive paperwork, and trust in a centralized authority. This creates several critical issues:

- High fees and slow processing times that exclude many users
- Dependency on a centralized entity, creating single points of failure
- Lack of transparency — users cannot verify what happens to their funds
- Exclusion of users without credit history or access to formal banking

1.2 How Does This Smart Contract Solve It?

This project implements a decentralized lending protocol inspired by Compound Finance. By deploying two ERC20 tokens and a lending contract on Ethereum, users can:

- Deposit ERC20 tokens as collateral — locked trustlessly in the smart contract
- Borrow other tokens — up to a safe limit based on their deposited collateral
- Repay borrowed tokens — to restore borrowing capacity
- Withdraw collateral — once their debt is fully cleared or sufficient collateral remains

Key Insight: The collateral ratio of 150% (over-collateralization) ensures the protocol remains solvent at all times — you must always lock at least 1.5x the value you wish to borrow. This removes counterparty risk entirely: the code is the bank.

Why Decentralized Lending Matters

Traditional Bank → Requires paperwork, credit score, days of processing, and trust in one entity.

Mini Compound → Instant, permissionless, transparent, and governed entirely by immutable code.

2. Code Explanation

2.1 MockERC20.sol

MockERC20 is a minimal ERC20 token used for testing. It inherits from OpenZeppelin's battle-tested ERC20 implementation, saving us from writing token logic from scratch.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
contract MockERC20 is ERC20 {
    constructor(
        string memory name,
        string memory symbol,
        uint256 initialSupply
    ) ERC20(name, symbol) {
        _mint(msg.sender, initialSupply);
    }
}
```

Part	Explanation
pragma solidity ^0.8.20	Specifies the Solidity compiler version. ^ means this version or higher (minor).
import "@openzeppelin/..."	Imports OpenZeppelin's battle-tested ERC20 implementation.
contract MockERC20 is ERC20	Our contract inherits all ERC20 functionality (transfer, approve, allowance, etc.).
constructor(name, symbol, initialSupply)	Called once at deployment. Sets token name, symbol, and mints the initial supply to the deployer.
_mint(msg.sender, initialSupply)	Creates tokens and assigns them to the deployer. Used for testing.

We need two ERC20 tokens for the protocol: one as collateral (COL) and one as the borrowable token (BRW). MockERC20 lets us create both easily in tests and local deployment.

2.2 LendingProtocol.sol

This is the core smart contract. It manages collateral deposits, borrowing limits, repayments, and withdrawals entirely on-chain.

State Variables

```
IERC20 public collateralToken;
IERC20 public borrowToken;
uint256 public collateralRatio = 150;
mapping(address => uint256) public collateralBalance;
mapping(address => uint256) public borrowedBalance;
```

Variable	Explanation
collateralToken	Reference to the ERC20 token users deposit as collateral (COL).
borrowToken	Reference to the ERC20 token users can borrow (BRW).
collateralRatio = 150	Over-collateralization ratio of 150%. To borrow 100 tokens, you must deposit 150 tokens. Protects the protocol from insolvency.
collateralBalance	Tracks how much collateral each user has deposited. Acts as an on-chain ledger: address → amount.
borrowedBalance	Tracks how much each user has currently borrowed.

Function: depositCollateral(uint256 amount)

```
function depositCollateral(uint256 amount) external {
    require(amount > 0, "Amount must be > 0");
    collateralToken.transferFrom(msg.sender, address(this), amount);
    collateralBalance[msg.sender] += amount;
}
```

Part	Explanation
external	Can only be called from outside the contract — more gas efficient than public.
require(amount > 0, ...)	Validation check. Reverts the transaction with an error message if condition fails.
transferFrom(...)	Moves tokens FROM the user TO the contract. Requires a prior approve() call from the user.
collateralBalance[msg.sender] += amount	Updates the user's balance in the internal on-chain ledger.

Function: borrow(uint256 amount)

```
function borrow(uint256 amount) external {
    require(amount > 0, "Amount must be > 0");
    uint256 maxBorrow = (collateralBalance[msg.sender]*100) / collateralRatio;
    require(borrowedBalance[msg.sender]+amount <= maxBorrow, "Not enough collateral");
    borrowedBalance[msg.sender] += amount;
    borrowToken.transfer(msg.sender, amount);
}
```

Part	Explanation
maxBorrow = (collateral*100)/150	Calculates max borrowable amount. If collateral = 150, maxBorrow = 100.
borrowedBalance+amount <= maxBorrow	Ensures total borrowed does not exceed the safe limit.
borrowToken.transfer(msg.sender, amount)	Sends borrow tokens directly to the user. No approval needed — contract already owns these tokens.

Example: Deposit 150 COL → maxBorrow = 100 BRW. Attempt to borrow 101 → transaction reverts with 'Not enough collateral'.

Function: repay(uint256 amount)

```
function repay(uint256 amount) external {
    require(amount > 0);
    borrowToken.transferFrom(msg.sender, address(this), amount);
    borrowedBalance[msg.sender] -= amount;
}
```

Part	Explanation
transferFrom(...)	Takes BRW tokens back from the user. Requires prior approval.
borrowedBalance -= amount	Reduces the debt. In Solidity 0.8+, this automatically reverts on underflow (prevents over-repaying).

Function: withdrawCollateral(uint256 amount)

```
function withdrawCollateral(uint256 amount) external {
    require(collateralBalance[msg.sender] >= amount, "Insufficient collateral");
    uint256 remaining = collateralBalance[msg.sender] - amount;
    uint256 required = (borrowedBalance[msg.sender] * collateralRatio) / 100;
    require(remaining >= required || borrowedBalance[msg.sender]==0, "Loan still active");
    collateralBalance[msg.sender] -= amount;
    collateralToken.transfer(msg.sender, amount);
}
```

Part	Explanation
remaining = collateral - amount	Calculates how much collateral would remain after the withdrawal.
required = borrowed * 150 / 100	Minimum collateral needed to back the existing debt at the 150% ratio.
remaining >= required OR borrowed == 0	Safety check: no debt exists, or the remaining collateral still fully backs the debt.
collateralToken.transfer(msg.sender, amount)	Returns collateral tokens to the user.

2.3 deploy.js

The deployment script creates both ERC20 tokens, deploys the LendingProtocol, and pre-funds it with 100,000 BRW tokens so users can borrow immediately.

```
const collateral = await Token.deploy("Collateral", "COL", parseEther("1000000"));
const borrow = await Token.deploy("Borrow", "BRW", parseEther("1000000"));
const lending = await Lending.deploy(collateral.target, borrow.target);
await borrow.transfer(lending.target, parseEther("100000"));
```

Part	Explanation
parseEther("1000000")	Converts 1,000,000 to wei (18 decimals). ERC20 tokens use the same unit system.
collateral.target	The deployed contract address — passed to LendingProtocol so it knows which token to use.
borrow.transfer(lending.target, ...)	Pre-funds the lending contract with 100,000 BRW so users can borrow from day one.

2.4 hardhat.config.js

```
module.exports = {
  solidity: "0.8.20",
  networks: {
    sepolia: {
      url: SEPOLIA_RPC_URL,
      accounts: [PRIVATE_KEY],
    },
  },
};
```

Part	Explanation
solidity: "0.8.20"	Specifies the Solidity compiler version for Hardhat to use.
networks.sepolia	Configuration for Sepolia testnet deployment.
SEPOLIA_RPC_URL	RPC endpoint from Infura or Alchemy. Loaded from .env for security.
PRIVATE_KEY	Deployer wallet private key. Loaded from .env — never hardcoded in source.

2.5 App.jsx (Frontend)

The React frontend uses ethers.js v6 to connect to MetaMask and interact with all four contract functions. It reads balances using read-only provider calls, and writes transactions using a signer.

Function	Description
connectWallet()	Requests MetaMask accounts via <code>eth_requestAccounts</code> . Wraps <code>window.ethereum</code> as a <code>BrowserProvider</code> .
loadBalances()	Creates a read-only contract instance and calls <code>collateralBalance()</code> and <code>borrowedBalance()</code> view functions.
deposit()	Two transactions: (1) <code>approve()</code> on COL token, (2) <code>depositCollateral()</code> on lending contract.
borrow()	Single transaction: calls <code>borrow()</code> on lending contract.
repay()	Two transactions: (1) <code>approve()</code> on BRW token, (2) <code>repay()</code> on lending contract.
withdraw()	Single transaction: calls <code>withdrawCollateral()</code> on lending contract.

The two-step approve + action pattern is mandatory because ERC20 tokens do not allow contracts to pull tokens without prior permission — this is a core security feature of the ERC20 standard enforced at the protocol level.

3. Smart Contract Flow Chart

The diagram below illustrates the complete user interaction lifecycle with the LendingProtocol smart contract.

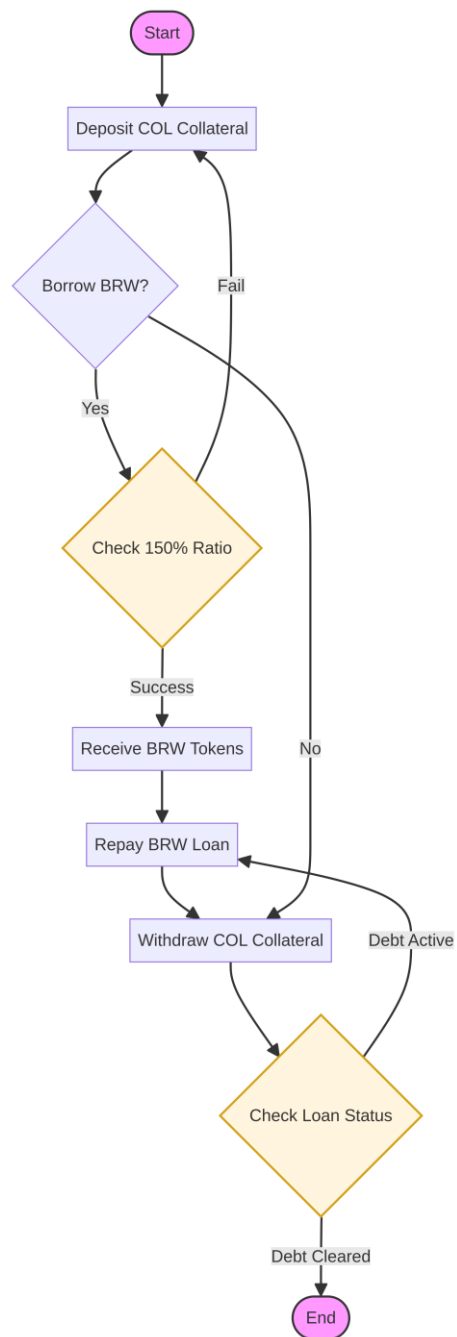


Figure 1 — Complete LendingProtocol interaction flow: deposit → borrow → repay → withdraw

4. Test Cases

All tests were written using Mocha + Chai and run with the Hardhat testing framework. The test file (Lending.js) covers the three core scenarios of the protocol.

Running the Tests

```
cd backend
npx hardhat test
```

```
silent_root@pop-os:~$ cd ~/mini-compound/backend
silent_root@pop-os:~/mini-compound/backend$ npx hardhat test
WARNING: You are currently using Node.js v18.20.8, which is not supported by Hardhat. This can lead to unexpected behavior. See https://v2.hardhat.org/nodejs-versions

  o injected env (2) from .env // tip: = enable debugging ( debug: true )

Lending Protocol
  ✓ Should deposit collateral
  ✓ Should borrow token
  ✓ Should repay loan

3 passing (402ms)
silent_root@pop-os:~/mini-compound/backend$
```

Terminal output: 3 passing (402ms) — all test cases confirmed

Test Results Summary

Test Case	What It Verifies	Result
Should deposit collateral	User approves 100 COL and deposits it. collateralBalance correctly records 100.	✓PASS
Should borrow token	User deposits 150 COL (maxBorrow = 100 BRW), borrows 50 BRW. borrowedBalance = 50.	✓PASS
Should repay loan	User repays 50 BRW. borrowedBalance returns to 0.	✓PASS

Test 1: Should deposit collateral

```
await collateral.connect(user)
  .approve(lending.target, ethers.parseEther("100"));
await lending.connect(user)
  .depositCollateral(ethers.parseEther("100"));
expect(await lending.collateralBalance(user.address))
  .to.equal(ethers.parseEther("100"));
```

Verifies that after approving and depositing 100 COL tokens, the on-chain collateralBalance mapping is updated correctly.

Test 2: Should borrow token

```
await collateral.connect(user).approve(lending.target, ethers.parseEther("150"));
await lending.connect(user).depositCollateral(ethers.parseEther("150"));
await lending.connect(user).borrow(ethers.parseEther("50"));
expect(await lending.borrowedBalance(user.address))
  .to.equal(ethers.parseEther("50"));
```

Deposits 150 COL (which allows borrowing up to 100 BRW), then borrows 50 BRW. Confirms that borrowedBalance accurately reflects the debt.

Test 3: Should repay loan

```
await borrow.connect(user).approve(lending.target, ethers.parseEther("50"));
await lending.connect(user).repay(ethers.parseEther("50"));
expect(await lending.borrowedBalance(user.address))
    .to.equal(0);
```

Approves and repays the 50 BRW debt. Confirms that borrowedBalance correctly returns to zero after full repayment.

5. Transaction Logs

All transactions were first executed on Hardhat Local Network (for development and testing) and subsequently on the Sepolia Ethereum Testnet for the bonus deployment.

5.1 Starting the Hardhat Node

```
cd backend
npx hardhat node
```

```
silent_root@pop-os:~/mini-compound/backend$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944abac478cded5efcae784d7bf4f2ff80

Account #1: 0x70997970c51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900Fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bfe6B2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267D07C367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9905007D1a55bC2695C58ba16FB37d81980A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA7404e5726554D8057FA54763abd0C3a0aa9 (10000 ETH)
Private Key: 0x92db14e403b33dfe3d3f233f83df3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e

Account #7: 0x14dc79964da2C08B23608B3D3cc7Ca32103d9955 (10000 ETH)
Private Key: 0x4bbb85ce377467afe5d46f04f221813b2bb87f24d81f60f1fcd8f7c7cf4356

Account #8: 0x23618e81E3f5cdf7f543C3d05f7FBC0aBf5821E8f (10000 ETH)
Private Key: 0xdbda1821b80551c9d65939329250298aa3472ba22f6ea921c0cf5d620ea67b97

Account #9: 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (10000 ETH)
Private Key: 0x2a871d0789f7d79848a013d4936a73bf4cc922c825d33c1cf7073dff6d409c6

Account #10: 0xBcd4042DE499D14e55001Ccb824a551F3b954096 (10000 ETH)
Private Key: 0xf214f2b2cd398c806f84e317254e0f0b801d0643303237d97a22a48e01628897

Account #11: 0x701bE63f3384f5fb98995898A86002Fb2426c5788 (10000 ETH)
Private Key: 0x701b015bbdfb9de65240bc28bd21bbc0d996645a3dd57e7b12bc2bdf6f192c82

Account #12: 0xFAB80ac9d6808445fB7357272F202C5651694a (10000 ETH)
Private Key: 0xa267530f49f8280200edf313ee7af6b827f2a8bce2897751d06a843f644967b1

Account #13: 0x1C8d3b2770909D4e10f157cABC84C7264073C9Ec (10000 ETH)
Private Key: 0x47c99abed3324a2707c28affff1267e45918ec8c3f20b8aa892e8b065d2942dd

Account #14: 0xdF3e18d648C6A983f673Ab319CCaE4f1a57C7097 (10000 ETH)
Private Key: 0xc526ee95bf44d8fc405a158bb884d9d1238d99f0612e9f33d006bb0789009aaa

Account #15: 0xcd3B766CCDD6AE721141F452C550Ca635964ce71 (10000 ETH)
Private Key: 0x0166f546bab6da521a8369cab0cc5d2b9e46670292d85c875ee9ec20e84ffb61

Account #16: 0x25468D3c84621e976D8185a91A922aE77ECEc30 (10000 ETH)
Private Key: 0xea0c44ac03bff858b476bba40716402b03e41b8e97e276d1baec7c37d42484a0

Account #17: 0xbDA5747bFD65F08deb54cb465e887D40e51B197E (10000 ETH)
Private Key: 0x689af8ef8a6c051a91ad287602527f3af2fe9f6501a7ac4b061667b5a93e037fd

Account #18: 0xdD2FD4581271e230360230F9337D5c04308f44C0 (10000 ETH)
Private Key: 0xde9be8858da4a475276426320d5e9262ecfc3ba460bfac56360bfa6c4c28b4ee0

Account #19: 0x8626f69402eb28930eFb4ceF4982d1F2C9C1199 (10000 ETH)
Private Key: 0xdf57089febbacf7ba0bc227dafbffa9fc08a93fdc68e1e42411a14efc723656e

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.
```

Hardhat node started — 20 test accounts each pre-funded with 10,000 ETH

```
Account #10: 0xBcd4042DE499D14e55001Ccb824a551F3b954096 (10000 ETH)
Private Key: 0xf214f2b2cd398c806f84e317254e0f0b801d0643303237d97a22a48e01628897

Account #11: 0x701bE63f3384f5fb98995898A86002Fb2426c5788 (10000 ETH)
Private Key: 0x701b015bbdfb9de65240bc28bd21bbc0d996645a3dd57e7b12bc2bdf6f192c82

Account #12: 0xFAB80ac9d6808445fB7357272F202C5651694a (10000 ETH)
Private Key: 0xa267530f49f8280200edf313ee7af6b827f2a8bce2897751d06a843f644967b1

Account #13: 0x1C8d3b2770909D4e10f157cABC84C7264073C9Ec (10000 ETH)
Private Key: 0x47c99abed3324a2707c28affff1267e45918ec8c3f20b8aa892e8b065d2942dd

Account #14: 0xdF3e18d648C6A983f673Ab319CCaE4f1a57C7097 (10000 ETH)
Private Key: 0xc526ee95bf44d8fc405a158bb884d9d1238d99f0612e9f33d006bb0789009aaa

Account #15: 0xcd3B766CCDD6AE721141F452C550Ca635964ce71 (10000 ETH)
Private Key: 0x0166f546bab6da521a8369cab0cc5d2b9e46670292d85c875ee9ec20e84ffb61

Account #16: 0x25468D3c84621e976D8185a91A922aE77ECEc30 (10000 ETH)
Private Key: 0xea0c44ac03bff858b476bba40716402b03e41b8e97e276d1baec7c37d42484a0

Account #17: 0xbDA5747bFD65F08deb54cb465e887D40e51B197E (10000 ETH)
Private Key: 0x689af8ef8a6c051a91ad287602527f3af2fe9f6501a7ac4b061667b5a93e037fd

Account #18: 0xdD2FD4581271e230360230F9337D5c04308f44C0 (10000 ETH)
Private Key: 0xde9be8858da4a475276426320d5e9262ecfc3ba460bfac56360bfa6c4c28b4ee0

Account #19: 0x8626f69402eb28930eFb4ceF4982d1F2C9C1199 (10000 ETH)
Private Key: 0xdf57089febbacf7ba0bc227dafbffa9fc08a93fdc68e1e42411a14efc723656e

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.
```

Accounts #10–19 listed with public/private key pairs

5.2 Local Deployment

```
npx hardhat run scripts/deploy.js --network localhost
```

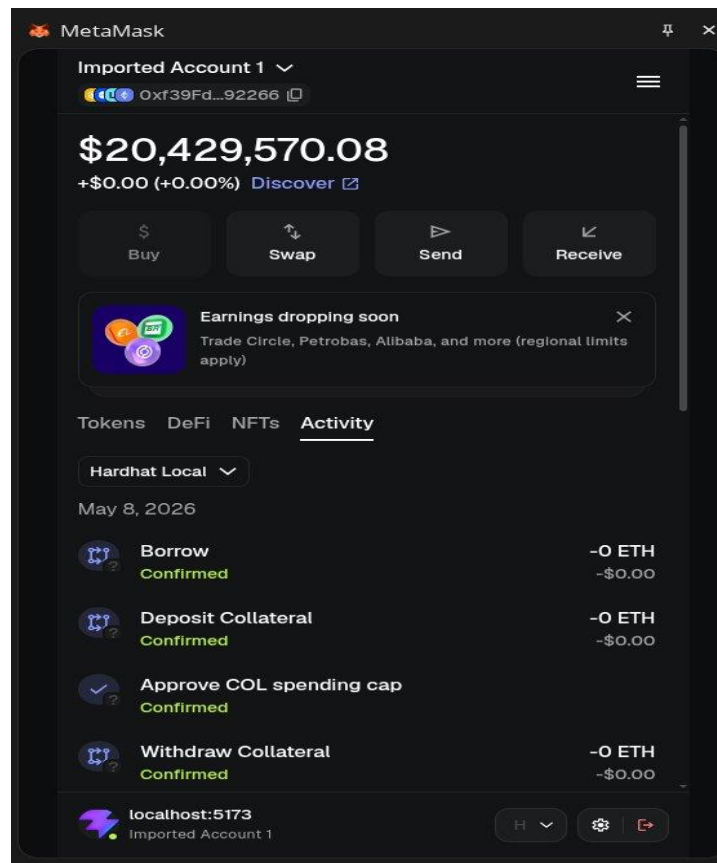
```
silent_root@pop-os:~/mini-compound/backend$ npx hardhat run scripts/deploy.js --network localhost
WARNING: You are currently using Node.js v22.22.2, which is not supported by Hardhat. This can lead to unexpected behavior. See https://hardhat.org/nodejs-versions

Collateral Token: 0x5FC8d32690cc91D4c39d9d3abcBD16989F875707
Borrow Token: 0x0165878A594ca255338adfa4d48449f69242Eb8F
Lending Protocol: 0xa513E6E4b8f2a923D98304ec87F64353C4D5C853
silent_root@pop-os:~/mini-compound/backend$
```

Successful local deployment — all three contract addresses printed to terminal

Contract	Local Address
Collateral Token (COL)	0x5FC8d32690cc91D4c39d9d3abcBD16989F875707
Borrow Token (BRW)	0x0165878A594ca255338adfa4d48449f69242Eb8F
Lending Protocol	0xa513E6E4b8f2a923D98304ec87F64353C4D5C853

5.3 MetaMask Activity Log

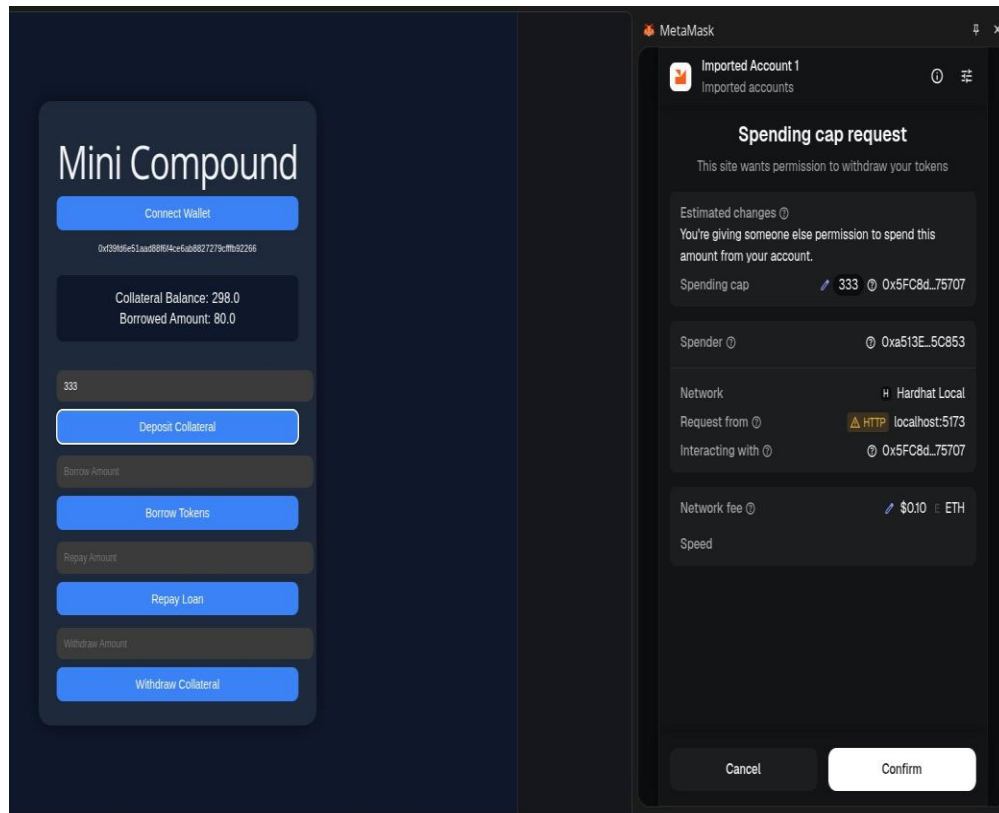


MetaMask Activity tab — all four transaction types confirmed on Hardhat Local (May 8, 2026)

#	Transaction	Description	Status
1	Approve COL spending cap	User allows LendingProtocol to spend their COL tokens before deposit.	✓ Confirmed
2	Deposit Collateral	Transfers COL tokens from user to contract. collateralBalance increases.	✓ Confirmed
3	Borrow	Contract sends BRW tokens to user. borrowedBalance increases.	✓ Confirmed
4	Withdraw Collateral	Returns COL tokens to user after debt safety check passes.	✓ Confirmed

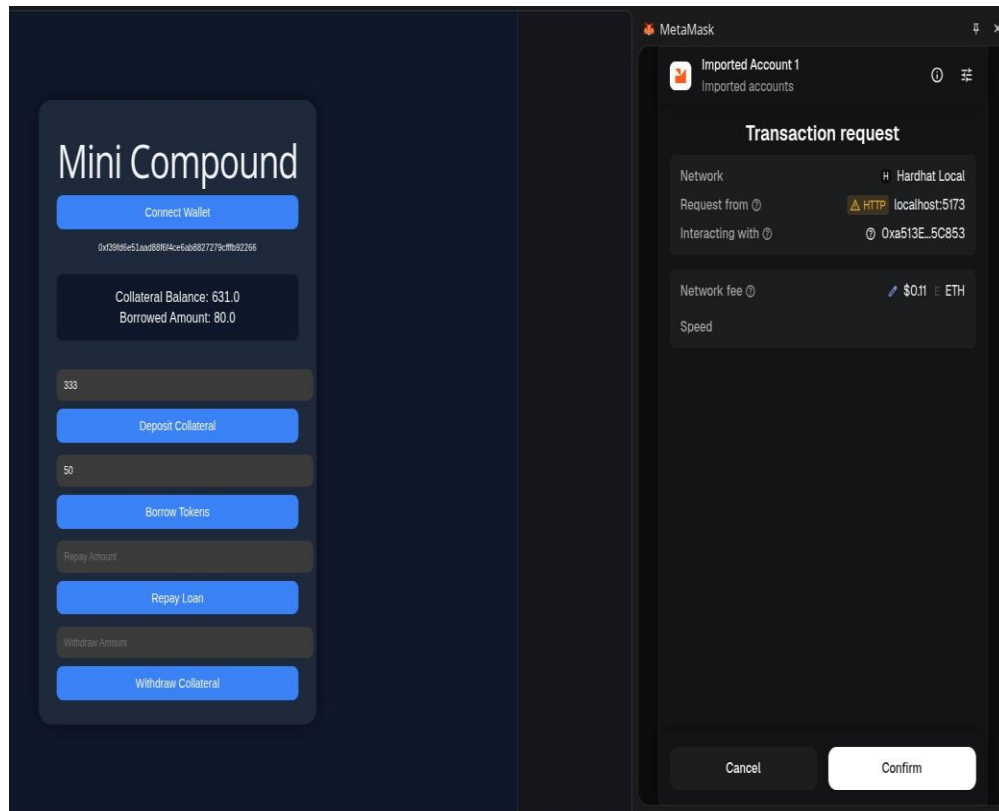
5.4 Detailed Transaction Screenshots

Deposit Collateral — Approve Step



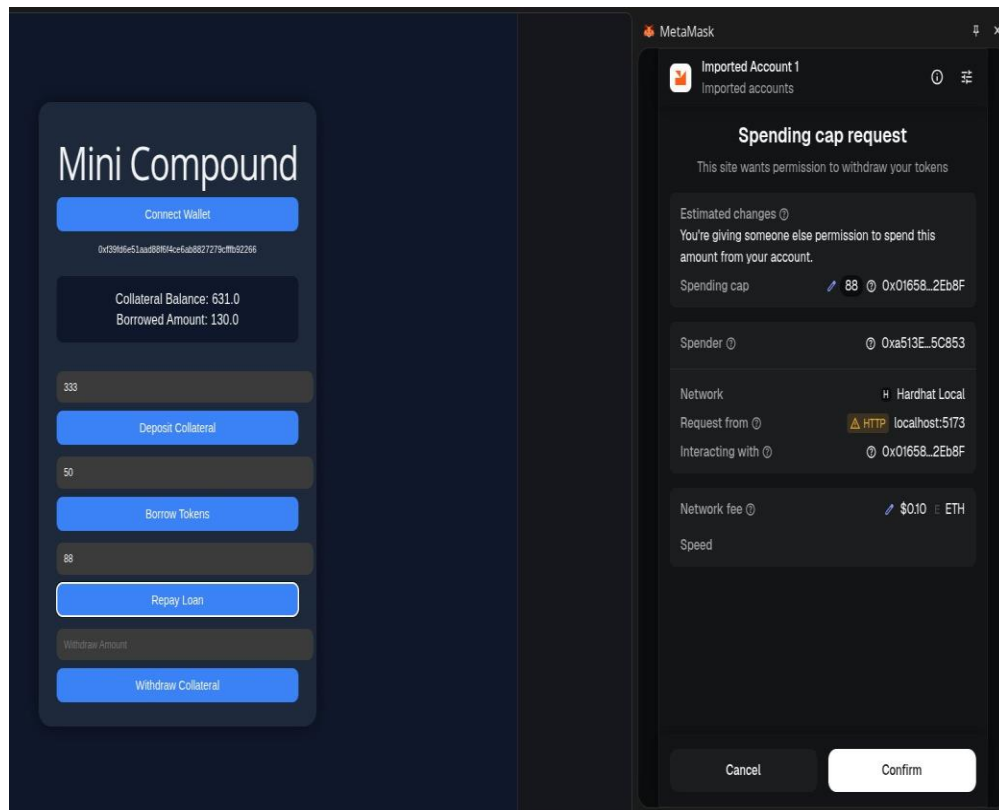
MetaMask spending cap request: giving LendingProtocol (0xa513E...5C853) permission to spend 333 COL tokens

Deposit Collateral — Confirm Step



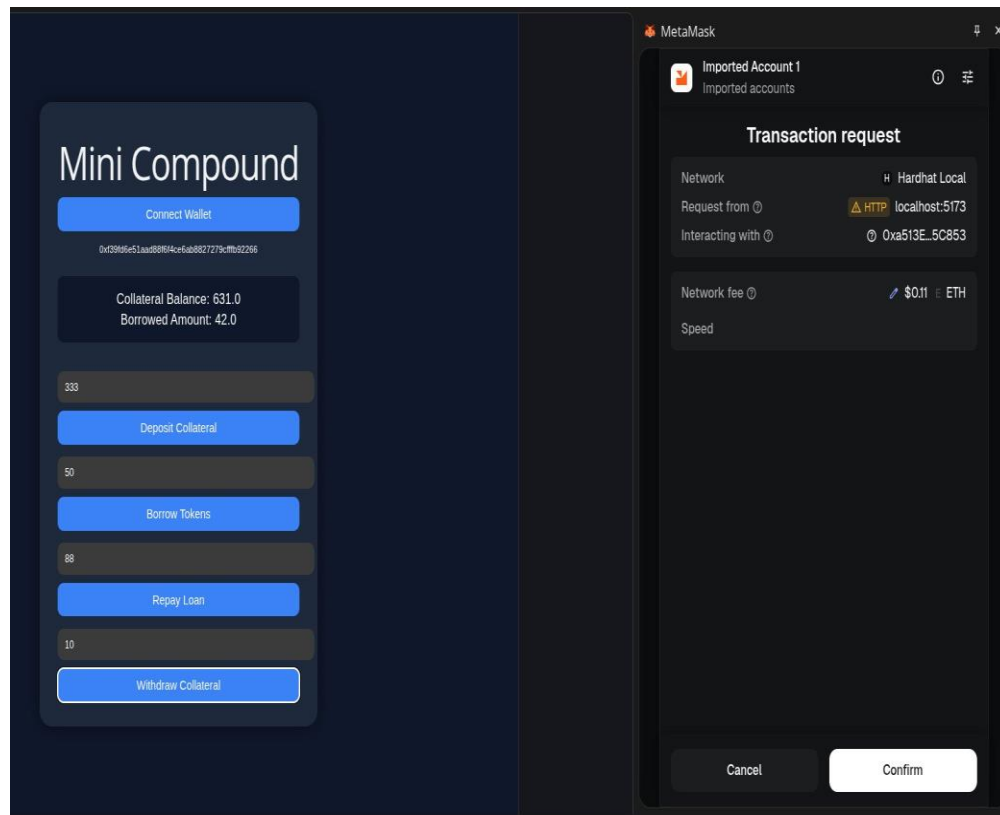
MetaMask transaction request to depositCollateral() — Network: Hardhat Local, fee: \$0.11 ETH

Repay Loan — Approve Step



Spending cap request for 88 BRW tokens (0x01658...2Eb8F) before calling repay()

Withdraw Collateral

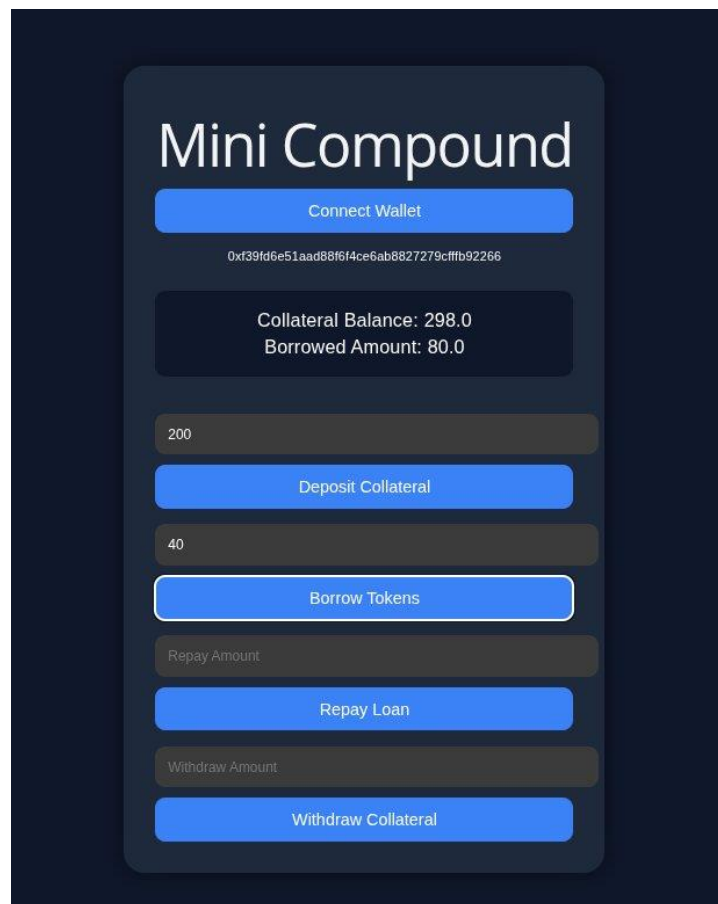


Transaction request to withdrawCollateral(10) — Collateral: 631.0, Borrowed: 42.0

6. Bonus — Frontend UI

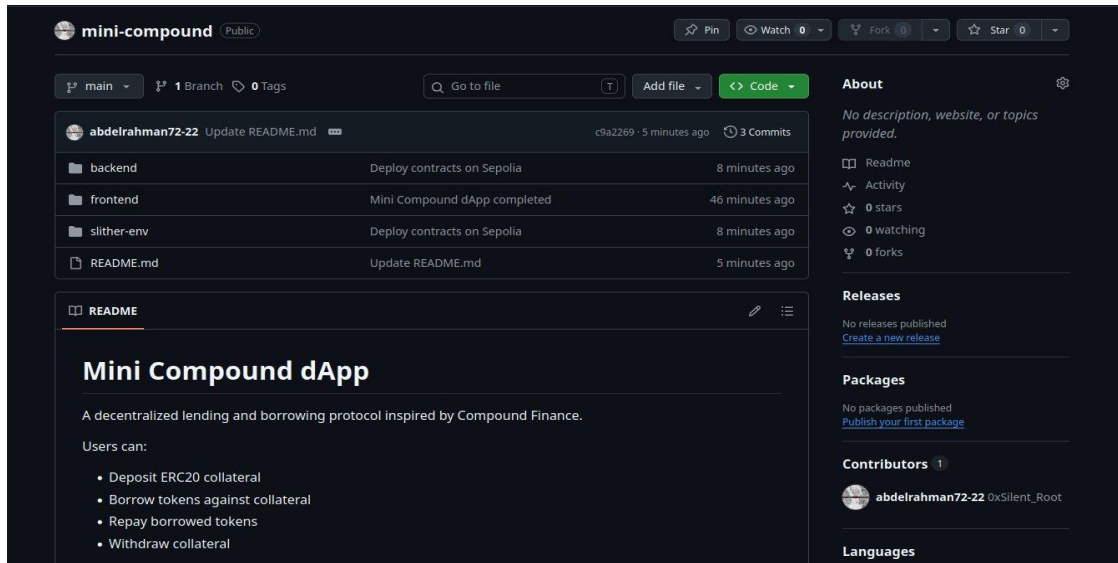
A complete React frontend was built using Vite + ethers.js v6 to interact with the deployed smart contracts via MetaMask.

Category	Details
Framework	React with useState hooks
Blockchain Library	ethers.js v6
Wallet	MetaMask (window.ethereum BrowserProvider)
Networks	Hardhat Local + Sepolia Testnet
Features	Connect wallet, live balance updates, deposit, borrow, repay, withdraw



Full UI — wallet connected, Collateral Balance: 298.0, Borrowed: 80.0, with all four action buttons

GitHub Repository



github.com/abdelrahman72-22/mini-compound — 3 commits, 3 folders (backend, frontend, slither-env)

7. Bonus — Slither Security Analysis

Slither is a static analysis framework for Solidity developed by Trail of Bits. It scans smart contracts for known vulnerability patterns without executing them.

```
slither . --solc-remaps "@openzeppelin=node_modules/@openzeppelin"
```

Result: Slither analyzed 9 contracts with 101 detectors and found 16 results.

```
(slither-env) silent_root@pop-os:~/mini-compound/backend$ slither . --solc-remaps "@openzeppelin=node_modules/@openzeppelin"
'npx hardhat clean' running (wd: /home/silent_root/mini-compound/backend)
'npx hardhat clean --global' running (wd: /home/silent_root/mini-compound/backend)
'npx hardhat compile --force' running (wd: /home/silent_root/mini-compound/backend)
INFO:Detectors:
Detector: unchecked-transfer
LendingProtocol.depositCollateral(uint256) (contracts/LendingProtocol.sol#24-35) ignores return value by collateralToken.transferFrom(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#28-32)
LendingProtocol.borrow(uint256) (contracts/LendingProtocol.sol#37-53) ignores return value by borrowToken.transferFrom(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#59-63)
LendingProtocol.repay(uint256) (contracts/LendingProtocol.sol#55-66) ignores return value by borrowToken.transferFrom(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#59-63)
LendingProtocol.withdrawCollateral(uint256) (contracts/LendingProtocol.sol#68-91) ignores return value by collateralToken.transfer(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#90)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unchecked-transfer
INFO:Detectors:
Detector: shadowing-local
MockERC20.constructor(string,string,uint256).name (contracts/MockERC20.sol#9) shadows:
- ERC20.name() (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#52-54) (function)
- IERC20Metadata.name() (node_modules/@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol#15) (function)
MockERC20.constructor(string,string,uint256).symbol (contracts/MockERC20.sol#10) shadows:
- ERC20.symbol() (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#60-62) (function)
- IERC20Metadata.symbol() (node_modules/@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol#20) (function)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#local-variable-shadowing
INFO:Detectors:
Detector: reentrancy-benign
Reentrancy in LendingProtocol.depositCollateral(uint256) (contracts/LendingProtocol.sol#24-35):
  External calls:
  - collateralToken.transferFrom(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#28-32)
  State variables written after the call(s):
  - collateralBalance[msg.sender] += amount (contracts/LendingProtocol.sol#34)
Reentrancy in LendingProtocol.repay(uint256) (contracts/LendingProtocol.sol#55-66):
  External calls:
  - borrowToken.transferFrom(msg.sender,address(this),amount) (contracts/LendingProtocol.sol#59-63)
  State variables written after the call(s):
  - borrowedBalance[msg.sender] -= amount (contracts/LendingProtocol.sol#65)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#reentrancy-vulnerabilities-3
INFO:Detectors:
Detector: pragma
4 different versions of Solidity are used:
```



```

4 different versions of Solidity are used:
- Version constraint >=0.8.4 is used by:
  ->=0.8.4 (node_modules/@openzeppelin/contracts/interfaces/draft-IERC6093.sol#4)
- Version constraint ^0.8.20 is used by:
  -^0.8.20 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#4)
  -^0.8.20 (node_modules/@openzeppelin/contracts/utils/Context.sol#4)
  -^0.8.20 (contracts/LendingProtocol.sol#2)
  -^0.8.20 (contracts/MockERC20.sol#2)
- Version constraint >=0.4.16 is used by:
  ->=0.4.16 (node_modules/@openzeppelin/contracts/token/ERC20/IERC20.sol#4)
- Version constraint >=0.6.2 is used by:
  ->=0.6.2 (node_modules/@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol#4)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#different-pragma-directives-are-used
INFO:Detectors:
Detector: solc-version
Version constraint >=0.8.4 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- MissingSideEffectsOnSelectorAccess
- AbiReencodingHeadOverflowWithStaticArrayCleanup
- DirtyByteArrayToStorage
- DataLocationChangeInInternalOverride
- NestedCalldataArrayAbiReencodingSizeValidation
- SignedImmutables.
It is used by:
- >=0.8.4 (node_modules/@openzeppelin/contracts/interfaces/draft-IERC6093.sol#4)
Version constraint ^0.8.20 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
- VerbatimInvalidDeduplication
- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- MissingSideEffectsOnSelectorAccess.
It is used by:
- ^0.8.20 (node_modules/@openzeppelin/contracts/token/ERC20/ERC20.sol#4)
- ^0.8.20 (node_modules/@openzeppelin/contracts/utils/Context.sol#4)
- ^0.8.20 (contracts/LendingProtocol.sol#2)
- ^0.8.20 (contracts/MockERC20.sol#2)
Version constraint >=0.4.16 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
- DirtyByteArrayToStorage
- ABIDecodeTwoDimensionalArrayMemory
- KeccakCaching
- EmptyByteArrayCopy
- DynamicArrayCleanup
- ImplicitConstructorCallvalueCheck
- TupleAssignmentMultiStackSlotComponents

```

Pragma and solc-version warnings from OpenZeppelin dependencies

```

- TupleAssignmentMultiStackSlotComponents
- MemoryArrayCreationOverflow
- privateCanBeOverridden
- SignedArrayStorageCopy
- ABIEncoderV2StorageArrayWithMultiSlotElement
- DynamicConstructorArgumentsClippedABIV2
- UninitializedFunctionPointerInConstructor_0.4.x
- IncorrectEventSignatureInLibraries_0.4.x
- ExpExponentCleanup
- NestedArrayFunctionCallDecoder
- ZeroFunctionSelector.
It is used by:
- >=0.4.16 (node_modules/@openzeppelin/contracts/token/ERC20/IERC20.sol#4)
Version constraint >=0.6.2 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
- MissingSideEffectsOnSelectorAccess
- AbiReencodingHeadOverflowWithStaticArrayCleanup
- DirtyByteArrayToStorage
- NestedCalldataArrayAbiReencodingSizeValidation
- ABIDecodeTwoDimensionalArrayMemory
- KeccakCaching
- EmptyByteArrayCopy
- DynamicArrayCleanup
- MissingEscapingInFormatting
- ArraySliceDynamicallyEncodedBaseType
- ImplicitConstructorCallvalueCheck
- TupleAssignmentMultiStackSlotComponents
- MemoryArrayCreationOverflow.
It is used by:
- >=0.6.2 (node_modules/@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol#4)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Detector: constable-states
LendingProtocol.collateralRatio (contracts/LendingProtocol.sol#11) should be constant
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-constant
INFO:Detectors:
Detector: immutable-states
LendingProtocol.borrowToken (contracts/LendingProtocol.sol#9) should be immutable
LendingProtocol.collateralToken (contracts/LendingProtocol.sol#8) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
INFO:Slither: analyzed (9 contracts with 101 detectors), 16 result(s) found
(slither-env) silent_root@pop-os:~/mini-compound/backends$

```

constable-states, immutable-states findings — final: 16 result(s) found

Findings & Fixes Summary

ID	Detector	Severity	Fix Applied
1	unchecked-transfer	Medium	Use SafeERC20.safeTransferFrom() instead of transferFrom()
2	reentrancy-benign	Low	Apply Checks-Effects-Interactions pattern
3	shadowing-local	Low	Rename constructor params with underscore prefix (_name, _symbol)
4	constable-states	Optimization	Declare collateralRatio as constant
5	immutable-states	Optimization	Declare collateralToken and borrowToken as immutable
6	pragma/solc-version	Info	Lock pragma to exact version: pragma solidity 0.8.20;

Finding 1: unchecked-transfer (Medium)

Issue: The return values of ERC20 transfer() and transferFrom() are ignored. Some non-standard ERC20 tokens return false on failure instead of reverting — silent failures would corrupt protocol state.

Fix — replace all raw transfer calls with OpenZeppelin SafeERC20:

```
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
using SafeERC20 for IERC20;
// Instead of: collateralToken.transferFrom(msg.sender, address(this), amount);
collateralToken.safeTransferFrom(msg.sender, address(this), amount);
```

Finding 2: reentrancy-benign (Low)

Issue: depositCollateral and repay perform external calls before updating state. A malicious token could re-enter the contract during transferFrom.

Fix — Checks-Effects-Interactions (CEI) pattern: always update state before making external calls:

```
function depositCollateral(uint256 amount) external {
    require(amount > 0, "Amount must be > 0");
    // EFFECTS first — update state
    collateralBalance[msg.sender] += amount;
    // INTERACTIONS last — external call
    collateralToken.safeTransferFrom(msg.sender, address(this), amount);
}
```

Finding 3: shadowing-local (Low)

Issue: Constructor parameters name and symbol in MockERC20 shadow parent ERC20 function names.

```
// Fix: add underscore prefix to parameters
constructor(string memory _name, string memory _symbol, uint256 initialSupply)
    ERC20(_name, _symbol) { mint(msg.sender, initialSupply); }
```

Findings 4 & 5: Gas Optimizations

```
uint256 public constant collateralRatio = 150; // constant: stored in bytecode
IERC20 public immutable collateralToken;      // immutable: set once in constructor
IERC20 public immutable borrowToken;
```

8. Bonus — Sepolia Testnet Deployment

The contracts were successfully compiled and deployed to the Sepolia Ethereum testnet — a public test network that mirrors Mainnet behavior without real ETH costs.

```
npx hardhat run scripts/deploy.js --network sepolia
```

```
silent_root@pop-os:~/mini-compound/backend$ npx hardhat run scripts/deploy.js --network sepolia
WARNING: You are currently using Node.js v18.20.8, which is not supported by Hardhat. This can lead to unexpe
cted behavior. See https://v2.hardhat.org/nodejs-versions

  ◊ injected env (2) from .env // tip: # suppress logs { quiet: true }
  Downloading compiler 0.8.20
  Compiled 7 Solidity files successfully (evm target: paris).
  ◊ injected env (0) from .env // tip: # multiple files { path: ['.env.local', '.env'] }
  Collateral Token: 0x300638cE9030215d6AeE7558A1B4Aee667Aa4Ce4
  Borrow Token: 0x91E9B2FcD3cC9D8ca7636F92D3D99FBBBb569e1D
  Lending Protocol: 0x636a1AE0ba076F16A04e82F0A3f0a6b266b307E5
```

Successful Sepolia deployment — 7 Solidity files compiled, three contract addresses issued

Deployed Contract Addresses (Sepolia)

Contract	Sepolia Address
Collateral Token (COL)	0x300638cE9030215d6AeE7558A1B4Aee667Aa4Ce4
Borrow Token (BRW)	0x91E9B2FcD3cC9D8ca7636F92D3D99FBBBb569e1D
Lending Protocol	0x636a1AE0ba076F16A04e82F0A3f0a6b266b307E5

Local vs Sepolia Comparison

Aspect	Local (Hardhat)	Sepolia Testnet
Network	localhost:8545	Sepolia RPC (Infura/Alchemy)
ETH	Fake (10,000 ETH each)	Real testnet ETH from faucet
Transactions	Instant	~12–15 seconds confirmation
Persistence	Lost on restart	Permanent on testnet
Verifiable	No	Yes (Etherscan Sepolia)
Cost	Free	Free (test ETH)

Environment Configuration (.env)

```
SEPOLIA_RPC_URL=https://sepolia.infura.io/v3/YOUR_PROJECT_KEY
PRIVATE_KEY=your wallet private key here # NEVER commit to GitHub
```

The private key is loaded via dotenv and never hardcoded in the configuration file. This is a security requirement for any production or testnet deployment.

